LAPORAN PRAKTIKUM KEAMANAN SIBER

Implementasi Enkripsi dan Dekripsi Simetris dengan AES-256-GCM



Nama Mahasiswa	Annisa Nur Shadrina
NIM	103072400134
Mata Kuliah	Keamanan Siber
Topik Praktikum	Symmetric Cryptography - AES-256-GCM

1. Tujuan Praktikum

Tujuan dari praktikum ini adalah untuk:
1. Memahami konsep dasar plaintext, ciphertext, dan kunci rahasia simetris.
2. Mengimplementasikan fitur enkripsi dan dekripsi teks berbasis CLI menggunakan algoritma AES-256-GCM.
3. Memahami dan memverifikasi peran Kunci Simetris, Salt (PBKDF2), dan Nonce/IV dalam menjamin kerahasiaan (*confidentiality*) dan integritas (*integrity*).
4. Menguji ketahanan program terhadap penanganan kunci yang salah atau ciphertext yang rusak tanpa menampilkan traceback mentah.

2. Penjelasan Peran Kunci, Salt, dan Nonce

A. Kunci Rahasia Simetris (AES-256 Key, 32 Byte):
Kunci simetris adalah parameter rahasia bersama yang digunakan oleh algoritma AES-256 untuk melakukan proses transformasi matematis (substitusi S-Box, pergeseran baris, pencampuran kolom, dan penambahan subkunci ronde). Pada kriptografi simetris, kunci yang sama wajib digunakan baik saat mengenkripsi plaintext maupun saat memulihkan (mendekripsi) ciphertext. Pada implementasi ini, kunci 256-bit (32 byte) tidak diinputkan mentah oleh pengguna, melainkan diturunkan secara aman dari kata sandi (*passphrase*) pengguna menggunakan fungsi derivasi kunci PBKDF2-HMAC-SHA256.

B. Salt (16 Byte Acak):
Salt adalah untaian byte acak yang dibangkitkan secara kriptografis (os.urandom(16)) dan dicampurkan bersama kata sandi pada saat proses derivasi kunci dengan PBKDF2-HMAC-SHA256 (sebanyak 200.000 iterasi). Salt berperan penting dalam:
• Mencegah serangan kamus terkomputasi sebelumnya (*Precomputed Dictionary Attack*) dan tabel pelangi (*Rainbow Table*).
• Memastikan bahwa kata sandi yang sama yang digunakan oleh dua pengguna (atau pada dua sesi berbeda) akan menghasilkan kunci enkripsi biner 32-byte yang sama sekali berbeda.
• Salt bukan rahasia sehingga aman digabungkan secara terbuka bersama ciphertext.

C. Nonce / Initialization Vector (12 Byte Acak):
Nonce (*Number used ONCE*) adalah nilai acak 12 byte (standar rekomendasi NIST SP 800-38D untuk AES-GCM) yang wajib unik untuk setiap kali operasi enkripsi dijalankan. Nonce berfungsi sebagai inisialisasi counter pada Galois/Counter Mode. Peran utama Nonce adalah:
• Memastikan sifat *semantic security* (probabilistic encryption): teks plaintext yang sama dan kata sandi yang sama tidak akan pernah menghasilkan ciphertext yang sama.
• Mencegah penyerang mendeteksi kesamaan isi pesan atau melakukan serangan pengulangan (*Replay Attack*).
• Nonce bukan rahasia dan disertakan di bagian depan payload untuk digunakan kembali saat proses dekripsi.

D. Struktur Payload Output Base64:
Agar data biner mentah dapat disalin dan ditampilkan di terminal tanpa risiko korupsi karakter,

seluruh komponen biner digabungkan menjadi satu kesatuan sebelum di-encode dengan Base64:
Payload = Salt (16 B) + Nonce (12 B) + Ciphertext + Authentication Tag (16 B)
Panjang minimum biner sebelum di-decode adalah 44 byte.

3. Fitur dan Alur Program

Aplikasi diimplementasikan dalam berkas 'Annisa Nur Shadrina_103072400134_AES.py' berbasis CLI terminal interaktif dengan 2 menu utama:
1. Menu 1 (Enkripsi): Menerima input string plaintext dan kata sandi, kemudian menghasilkan string ciphertext Base64.
2. Menu 2 (Dekripsi): Menerima input ciphertext Base64 dan kata sandi, kemudian memvalidasi integritas data dan mengembalikan plaintext asli.
3. Menu 0 (Keluar): Menutup program terminal secara aman.

Penanganan Error: Program secara eksplisit menangani eksepsi InvalidTag, ValueError, dan UnicodeDecodeError tanpa mencetak traceback error ke pengguna.

4. Hasil Pengujian 4 Skenario Wajib

Berikut adalah dokumentasi hasil pengujian langsung program terhadap keempat skenario wajib yang ditentukan dalam modul tugas:

No.	Skenario Pengujian	Hasil yang Diharapkan	Status Pengujian
1	Teks biasa: "Halo Informatika"	Teks hasil dekripsi identik dengan input asli.	SUKSES / PASSED (Identik 100%)
2	Teks dengan angka, simbol, dan Unicode: "Siber2026! #AES_GCM$ @Tel-U (NIM: 103072400134)"	Teks hasil dekripsi tetap identik secara utuh.	SUKSES / PASSED (Identik 100%)
3	Dekripsi dengan kata sandi salah: Kata sandi keliru diinputkan	Program menolak dekripsi dan menampilkan pesan kesalahan ramah.	SUKSES / PASSED (Ditolak oleh InvalidTag)
4	Ciphertext Base64 diubah/rusak: Karakter ciphertext dimodifikasi	Program menolak dekripsi dan tidak menghasilkan data sampah.	SUKSES / PASSED (Otentikasi tag gagal)

Log Output Eksekusi Terminal

```
=== LOG PENGUJIAN OTOMATIS (test_runner.py) ===

[SKENARIO 1: Teks Biasa]
Plaintext Input : Halo Informatika
Kata Sandi      : KatasandiRahasia123
```

```
Ciphertext (B64): gxqrYfPBjEwW4qtabtY49i6JjztLx7Yeps3jpVy6HOkYIXBOwGzY/K97ZFX5EKGjA71dIH72Ho4SC8bj
Hasil Dekripsi  : Halo Informatika
Status          : SUKSES (Teks hasil dekripsi identik)

[SKENARIO 2: Teks Angka, Simbol, Unicode]
Plaintext Input : Siber2026! #AES_GCM$ @Tel-U (NIM: 103072400134)
Kata Sandi      : P@ssw0rd_Kuat!
Ciphertext (B64):
60zjdqKyqnCMZSWCIYfQhxkSvcUgyafl7M3Z0YCxVML3GHrMhBeh89Z7YdgODGkrkfu6shaJepdPURwTMOHlssjUdL5xpQud6v89
0IuwJqFvn+MFXoCqZYloeJH9rL2P
Hasil Dekripsi  : Siber2026! #AES_GCM$ @Tel-U (NIM: 103072400134)
Status          : SUKSES (Teks hasil dekripsi identik)

[SKENARIO 3: Kata Sandi Salah]
Ciphertext Input: gxqrYfPBjEwW4qtabtY49i6JjztLx7Yeps3jpVy6HOkYIXBOwGzY/K97ZFX5EKGjA71dIH72Ho4SC8bj
Kata Sandi Salah: KataSandiSalahTotal
Hasil Tangkapan : InvalidTag (Dekripsi gagal: kata sandi atau ciphertext tidak sesuai.)
Status          : SUKSES (Program menolak dekripsi secara aman)

[SKENARIO 4: Ciphertext Rusak]
Ciphertext Rusak: gxqrYfPBjEwW4qtabtY4Ai6JjztLx7Yeps3jpVy6HOkYIXBOwGzY/K97ZFX5EKGjA71dIH72Ho4SC8bj
Kata Sandi      : KatasandiRahasia123
Hasil Tangkapan : InvalidTag (Dekripsi gagal: kata sandi atau ciphertext tidak sesuai.)
Status          : SUKSES (Program menolak ciphertext yang dimanipulasi)
```

Tangkapan Layar Pengujian (Screenshots)

[Gambar 1: Screenshot Fitur 1 - Enkripsi Teks]

```
PS C:\Users\LENOVO IP FLEX 5> cd "D:\KULIAH\SEMESTER 5\CYBERSEC\Enkripsi Simetris"
PS D:\KULIAH\SEMESTER 5\CYBERSEC\Enkripsi Simetris> python "Annisa Nur Shadrina_103072400134_AES.py"

=== ENKRIPSI SIMETRIS (AES-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 1
Masukkan plaintext: Halo Informatika
Masukkan kata sandi: Rahasia123
Ciphertext (Base64): jQJxfza+lH7TmbUFwUlGkJl1S4OhY+aZj55WWGTPLiTeE3oweKkoUPa13yBRK/+0/Te2+o7nhA4k9Vlv

=== ENKRIPSI SIMETRIS (AES-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: jQJxfza+lH7TmbUFwUlGkJl1S4OhY+aZj55WWGTPLiTeE3oweKkoUPa13yBRK/+0/Te2+o7nhA4k9Vlv
Masukkan kata sandi: Rahasia123
Plaintext: Halo Informatika
```

(Silakan lampirkan tangkapan layar terminal saat menjalankan menu 1 - Enkripsi teks)

[Gambar 2: Screenshot Fitur 2 - Dekripsi Teks Sukses & Gagal]

```
=== ENKRIPSI SIMETRIS (AES-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 1
Masukkan plaintext: Siber2026! #AES_GCM$ @Tel-U (NIM: 103072400134)
Masukkan kata sandi: Rahasia123
Ciphertext (Base64): gKxcMw1jtJcrkJLHD9Mj3xP0heHnYXE2EGQzyvZuLbsuFDi4epzywZM2z0qMXRGRRTNdasp5QXcEZoNR3e/9GueX+dMQCnvyP/f
sbfCE6BndDcvWt3dylEIarw==

=== ENKRIPSI SIMETRIS (AES-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: gKxcMw1jtJcrkJLHD9Mj3xP0heHnYXE2EGQzyvZuLbsuFDi4epzywZM2z0qMXRGRRTNdasp5QXcEZoNR3e/9GueX+dMQ
CnvyP/fsbfCE6BndDcvWt3dylEIarw==
Masukkan kata sandi: Rahasia123
Plaintext: Siber2026! #AES_GCM$ @Tel-U (NIM: 103072400134)
```

```
=== ENKRIPSI SIMETRIS (AES-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: gKxcMw1jtJcrkJLHD9Mj3xP0heHnYXE2EGQzyvZuLbsuFDi4epzywZM2z0qMXRGRRTNdasp5QXcEZoNR3e/9GueX+dMQ
CnvyP/fsbfCE6BndDcvWt3dylEIarw==
Masukkan kata sandi: PasswordSalah
Dekripsi gagal: kata sandi atau ciphertext tidak sesuai.

=== ENKRIPSI SIMETRIS (AES-GCM) ===
1. Enkripsi teks
2. Dekripsi teks
0. Keluar
Pilih menu: 2
Masukkan ciphertext Base64: gKxcMw1jtJcrkJLHD9Mj3xP0heHnYXE2EGQzyvZuLbsuFDi4epzywZM2z0qMXRGRRTNdasp5QXcEZoNR3e/9GueX+dMQ
CnvyP/fsbfCE6BndDcvWt3dyhksarw==
Masukkan kata sandi: Rahasia123
Dekripsi gagal: kata sandi atau ciphertext tidak sesuai.
```

(Silakan lampirkan tangkapan layar terminal saat menjalankan menu 2 - Dekripsi dengan password benar vs salah)

5. Jawaban Pertanyaan Refleksi

Pertanyaan 1: Mengapa kata sandi yang sama diperlukan ketika melakukan dekripsi?

Jawaban:

Kriptografi simetris (seperti AES) beroperasi menggunakan prinsip kunci bersama (shared secret key). Operasi matematis pada proses enkripsi dan dekripsi bersifat berbalikan (invers) dan hanya dapat ditutup serta dibuka oleh kunci biner yang sama persis. Pada implementasi ini, kunci 256-bit diturunkan melalui fungsi PBKDF2 dari kata sandi dan salt. Karena sifat fungsi hash kriptografis yang memiliki efek longsoran (avalanche effect), perbedaan sekecil satu karakter saja pada kata sandi akan menghasilkan kunci biner 32-byte yang berbeda secara drastis. Kunci yang keliru tidak akan mampu membalikkan permutasi AES dan akan langsung gagal saat validasi Authentication Tag pada mode GCM. Oleh karena itu, kata sandi yang sama persis adalah syarat mutlak untuk meregenerasi kunci simetris yang valid.

Pertanyaan 2: Mengapa ciphertext perlu dikonversi ke Base64 ketika ditampilkan sebagai teks?

Jawaban:

Hasil keluaran enkripsi AES-GCM (yang digabungkan dengan salt dan nonce) berupa rangkaian byte biner mentah (raw bytes) dengan rentang nilai 0x00 hingga 0xFF. Sebagian besar nilai biner tersebut bukan merupakan karakter teks yang dapat dicetak (non-printable characters), melainkan kode kontrol terminal (seperti NULL, newline, EOF, bell) dan byte yang tidak sesuai dengan format standar teks UTF-8 atau ASCII. Jika byte biner ini dipaksa dicetak langsung ke terminal atau disalin-tempel, data akan terpotong, mengalami korupsi nilai byte, atau menyebabkan kegagalan decoding. Base64 mengonversi setiap 3 byte biner menjadi 4 karakter ASCII yang aman (A-Z, a-z, 0-9, +, /, =), sehingga payload dapat ditampilkan di terminal, disalin ke clipboard, dan dikirimkan tanpa risiko kerusakan data.

Pertanyaan 3: Mengapa proses enkripsi dengan plaintext dan kata sandi yang sama dapat menghasilkan ciphertext berbeda?

Jawaban:

Hal ini disebabkan oleh penerapan Salt acak (16 byte) dan Nonce acak (12 byte) baru yang selalu dibangkitkan secara independen menggunakan 'os.urandom()' setiap kali fungsi enkripsi dijalankan:

1. Peran Salt Acak: Walaupun kata sandi yang diketik sama, salt yang berbeda menyebabkan fungsi PBKDF2 menghasilkan kunci AES 256-bit yang berbeda secara unik pada setiap eksekusi.
2. Peran Nonce Acak: Pada mode AES-GCM, nonce berfungsi sebagai titik awal counter blok. Nilai counter yang unik memastikan aliran kunci (keystream) yang dihasilkan selalu berbeda, sehingga ciphertext yang dihasilkan dari plaintext yang sama akan selalu berubah secara acak.

Sifat ini merupakan standar keamanan modern (Semantic Security / Probabilistic Encryption) untuk mencegah serangan analisis pola/frekuensi, mencegah penyerang mengetahui apakah dua pesan terenkripsi memiliki isi yang sama, serta mencegah serangan Rainbow Table dan Replay Attack.

6. Kesimpulan

1. Algoritma AES-256-GCM berhasil diimplementasikan secara optimal pada program terminal Python berbasis spesifikasi modul tugas.
2. Penggunaan PBKDF2-HMAC-SHA256 (200.000 iterasi) dengan salt 16-byte efektif melindungi kata sandi dari serangan kamus dan brute-force.
3. Penerapan nonce acak 12-byte dan tag otentikasi 16-byte pada mode GCM tidak hanya menjamin kerahasiaan (*confidentiality*) tetapi juga menjamin keaslian dan integritas (*integrity/authenticity*) data, di mana manipulasi sekecil apapun pada ciphertext akan langsung ditolak.
4. Program berhasil lulus 100% pada seluruh 4 skenario pengujian wajib dan menangani error secara elegan tanpa traceback mentah.

Code python :

```python
import base64
import hashlib
import os

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.exceptions import InvalidTag


def turunkan_kunci(kata_sandi: str, salt: bytes) -> bytes:
    """Kunci 32 byte untuk AES-256 dari kata sandi dan salt menggunakan PBKDF2-HMAC-SHA256."""
    return hashlib.pbkdf2_hmac(
        "sha256", kata_sandi.encode("utf-8"), salt, 200_000, dklen=32
    )


def enkripsi(teks: str, kata_sandi: str) -> str:
    """Input: dua teks (plaintext dan kata sandi).
    Output: satu teks ciphertext yang dikodekan dengan Base64.

    Format payload biner: salt (16 byte) + nonce (12 byte) + ciphertext & auth tag.
    """
    # 1. Buat salt acak 16 byte dan nonce acak 12 byte
    salt = os.urandom(16)
    nonce = os.urandom(12)

    # 2. Buat kunci AES-256 (32 byte) dengan turunkan_kunci()
    kunci = turunkan_kunci(kata_sandi, salt)

    # 3. Enkripsi teks UTF-8 menggunakan AESGCM (otomatis menyertakan authentication tag 16 byte)
    aesgcm = AESGCM(kunci)
    ciphertext_with_tag = aesgcm.encrypt(nonce, teks.encode("utf-8"), None)

    # 4. Gabungkan salt + nonce + ciphertext (termasuk tag)
    payload = salt + nonce + ciphertext_with_tag

    # 5. Encode gabungan payload dengan Base64, lalu kembalikan sebagai string
    return base64.b64encode(payload).decode("ascii")


def dekripsi(ciphertext_b64: str, kata_sandi: str) -> str:
    """Input: dua teks (ciphertext Base64 dan kata sandi).
```

```
    Output: plaintext asli atau membangkitkan exception yang ditangani oleh
pemanggil.
    """
    # 1. Decode ciphertext_b64 dari Base64 menjadi bytes
    try:
        data = base64.b64decode(ciphertext_b64, validate=True)
    except Exception as exc:
        raise ValueError("Format Base64 tidak valid.") from exc

    # Validasi panjang minimum: salt(16) + nonce(12) + tag(16) = 44 bytes
    if len(data) < 44:
        raise ValueError("Data ciphertext terlalu pendek atau rusak.")

    # 2. Pisahkan salt (16 byte), nonce (12 byte), dan sisa ciphertext beserta
tag
    salt = data[:16]
    nonce = data[16:28]
    ciphertext_with_tag = data[28:]

    # 3. Buat kembali kunci dengan turunkan_kunci()
    kunci = turunkan_kunci(kata_sandi, salt)

    # 4. Dekripsi AESGCM, lalu decode UTF-8 menjadi teks
    aesgcm = AESGCM(kunci)
    plaintext_bytes = aesgcm.decrypt(nonce, ciphertext_with_tag, None)
    return plaintext_bytes.decode("utf-8")


def main() -> None:
    while True:
        print("\n=== ENKRIPSI SIMETRIS (AES-GCM) ===")
        print("1. Enkripsi teks")
        print("2. Dekripsi teks")
        print("0. Keluar")
        pilihan = input("Pilih menu: ").strip()
        if pilihan == "0":
            print("Program selesai.")
            break
        if pilihan not in {"1", "2"}:
            print("Pilihan menu tidak valid.")
            continue

        pesan = input("Masukkan plaintext: " if pilihan == "1" else "Masukkan
ciphertext Base64: ")
        kata_sandi = input("Masukkan kata sandi: ")
```

```
        if not kata_sandi:
            print("Kata sandi tidak boleh kosong.")
            continue

        try:
            if pilihan == "1":
                hasil_b64 = enkripsi(pesan, kata_sandi)
                print("Ciphertext (Base64):", hasil_b64)
            else:
                hasil_plain = dekripsi(pesan, kata_sandi)
                print("Plaintext:", hasil_plain)
        except (InvalidTag, ValueError, UnicodeDecodeError):
            print("Dekripsi gagal: kata sandi atau ciphertext tidak sesuai.")
        except NotImplementedError as exc:
            print(exc)


if __name__ == "__main__":
    main()
```